

AWS CLOUD DEVSECOPS PLATFORM

Production-Grade CI/CD and Kubernetes Deployment on AWS

Project Report

Submitted by: Girish Mahajan

Department of Information Technology

Academic Year: 2026–2027

Abstract

This project implements an end-to-end Cloud DevSecOps platform for deploying a containerized Python Flask application on Amazon Web Services. Infrastructure is provisioned using Terraform, while GitHub Actions automates the CI/CD workflow. Docker is used for containerization, Trivy performs container vulnerability scanning, and Amazon ECR provides private container image storage. The application is deployed to Kubernetes using Amazon EKS, with health and readiness endpoints for workload reliability. Prometheus and Grafana provide monitoring and observability. The project demonstrates practical implementation of Infrastructure as Code, CI/CD, container security, Kubernetes orchestration, monitoring, troubleshooting, and cloud resource lifecycle management.

1. Introduction

Modern cloud applications require repeatable infrastructure, automated delivery, security controls, container orchestration, and operational visibility. Traditional manual deployment processes can introduce configuration drift, deployment errors, inconsistent environments, and security gaps. DevOps practices address these problems through automation and collaboration, while DevSecOps integrates security into the delivery lifecycle. This project combines AWS, Terraform, Docker, Kubernetes, GitHub Actions, Trivy, Prometheus, and Grafana into one practical workflow.

2. Problem Statement

Manual provisioning and deployment of applications can be slow and error-prone. Infrastructure may be configured differently between environments, container images may be deployed without security checks, and application health may not be visible after deployment. The project addresses these issues by automating infrastructure provisioning, image creation, vulnerability scanning, registry publishing, Kubernetes deployment, health checking, and monitoring.

3. Objectives

The objectives are to provision AWS infrastructure using Terraform; build and containerize a Flask application; automate CI/CD with GitHub Actions; introduce Trivy security scanning; store images in Amazon ECR; deploy the application to Amazon EKS; implement health and readiness checks; expose Prometheus metrics; monitor the environment using Prometheus and Grafana; and practice cost-aware AWS resource lifecycle management.

4. System Architecture

The solution follows the flow: Developer → GitHub → GitHub Actions → Docker build → Trivy scan → Amazon ECR → Amazon EKS → Kubernetes → Prometheus → Grafana. Terraform operates as the Infrastructure as Code layer and provisions the AWS resources required by the platform. Kubernetes manages application workloads through namespaces, deployments, pods, and services.

5. Technology Stack

AWS provides the cloud platform. Terraform provides Infrastructure as Code. Python Flask provides the application. Docker packages the application into a container. Amazon ECR stores container images. Amazon EKS provides managed Kubernetes. GitHub Actions provides CI/CD automation. Trivy performs container vulnerability scanning. Prometheus collects metrics and Grafana visualizes them. AWS CLI and kubectl provide command-line management.

6. Application Development

The Flask application is intentionally lightweight so that the project focuses on cloud and DevSecOps engineering rather than application complexity. The root endpoint returns a simple running message. The /health endpoint returns a healthy status and can be used for liveness or health checking. The /ready endpoint indicates that the application is ready to receive traffic. The /metrics endpoint exposes Prometheus-compatible metrics.

7. Docker Implementation

The Flask application is packaged using a Dockerfile. The image can be built using 'docker build -t cloud-devops-platform .' and tested locally with 'docker run -p 5000:5000 cloud-devops-platform'. Containerization provides a consistent application runtime and makes the same image suitable for local Kubernetes and AWS EKS deployment.

8. Infrastructure as Code with Terraform

Terraform is used to define and manage AWS infrastructure declaratively. The project includes networking resources, IAM configuration, ECR, EKS, worker nodes, security-related resources, and supporting logging/networking components. The standard workflow is terraform init, terraform validate, terraform plan, terraform apply, and terraform destroy. This makes infrastructure repeatable, version-controlled, and easier to clean up after testing.

9. CI/CD Pipeline

The CI/CD workflow begins when code is pushed to GitHub. GitHub Actions builds the Docker image, runs a Trivy security scan, authenticates to Amazon ECR, and pushes the image to the registry. This introduces automation and a security validation stage before the image is consumed by Kubernetes.

10. Container Security

Trivy is integrated into the pipeline to identify known vulnerabilities in container images. This provides an early feedback mechanism for operating-system and dependency vulnerabilities. AWS credentials are not stored in the repository, generated Terraform provider binaries are excluded through .gitignore, and Terraform state files are kept out of source control.

11. Kubernetes and Amazon EKS

The application is deployed to Kubernetes using declarative YAML manifests. The deployment defines the desired application workload, while the Service provides stable networking to the Pods. Health and readiness endpoints are used by Kubernetes to determine workload state. Amazon EKS provides the managed Kubernetes control plane and worker infrastructure required to run the application in AWS.

12. Monitoring and Observability

Prometheus is used to collect application and Kubernetes metrics. The Flask application exposes /metrics, allowing Prometheus to scrape application-level measurements. A ServiceMonitor can be used with the Prometheus Operator to discover the application metrics endpoint. Grafana provides dashboards and visualization for operational analysis.

13. Testing and Verification

The project was validated through application endpoint checks, Docker build and runtime testing, GitHub Actions execution, Trivy scanning, ECR image publishing, Kubernetes node and Pod checks, Service verification, and monitoring validation. Representative verification commands include kubectl get nodes, kubectl get pods -n cloud-devops, kubectl get svc -n cloud-devops, and Terraform plan/apply/destroy workflows.

14. Challenges and Solutions

A significant GitHub issue occurred when the generated .terraform directory contained an AWS provider binary larger than GitHub's 100 MB file limit. The generated provider directory was removed from Git tracking and added to .gitignore, while terraform.lock.hcl was retained. Another cleanup issue occurred because Amazon ECR does not delete a repository that still contains images unless forced; the repository was intentionally retained for future development. A temporary DNS/network failure also interrupted EKS deletion, but direct AWS CLI tests later confirmed

that DNS and AWS connectivity were functioning.

15. Cost Management

AWS resources can generate charges even when a project is used only for learning. EKS, worker nodes, NAT gateways, load balancers, storage, and CloudWatch resources therefore need lifecycle management. After testing, Terraform destroy was used to remove temporary infrastructure. The ECR repository was retained to preserve the project image for future work. This demonstrates awareness of cloud cost control and resource lifecycle management.

16. Results

The completed implementation demonstrates an end-to-end Cloud DevSecOps workflow from source control and automated CI/CD through container security scanning, ECR image storage, Kubernetes deployment, health checks, and monitoring. The project provides practical experience with AWS infrastructure, Terraform, Docker, Kubernetes, EKS, ECR, GitHub Actions, Trivy, Prometheus, Grafana, troubleshooting, and cost-aware cloud operations.

17. Future Scope

Potential improvements include Argo CD GitOps deployment, AWS Load Balancer Controller, HTTPS using AWS Certificate Manager, Route 53 integration, Horizontal Pod Autoscaling, Alertmanager notifications, centralized logging, AWS Secrets Manager, Kubernetes Network Policies, blue/green and canary deployments, Terraform remote state, and multi-environment infrastructure.

18. Conclusion

The AWS Cloud DevSecOps Platform demonstrates how modern cloud-native applications can be provisioned, secured, containerized, deployed, and monitored using an integrated set of DevOps and DevSecOps technologies. The project emphasizes practical engineering, automation, troubleshooting, observability, and cost management rather than isolated tool usage. It provides a strong foundation for further GitOps, security, reliability, and cloud architecture improvements.

19. References

Official documentation used for implementation includes AWS documentation, Terraform documentation, Docker documentation, Kubernetes documentation, GitHub Actions documentation, Trivy documentation, Prometheus documentation, and Grafana documentation.

Appendix A — Key Commands

```
terraform init
terraform validate
terraform plan
terraform apply
terraform destroy
aws eks update-kubeconfig --region ap-south-1 --name cloud-devops-platform
kubectl get nodes
kubectl get pods -n cloud-devops
kubectl get svc -n cloud-devops
kubectl apply -f kubernetes/
docker build -t cloud-devops-platform .
```

```
docker run -p 5000:5000 cloud-devops-platform
```

```
trivy image cloud-devops-platform:latest
```